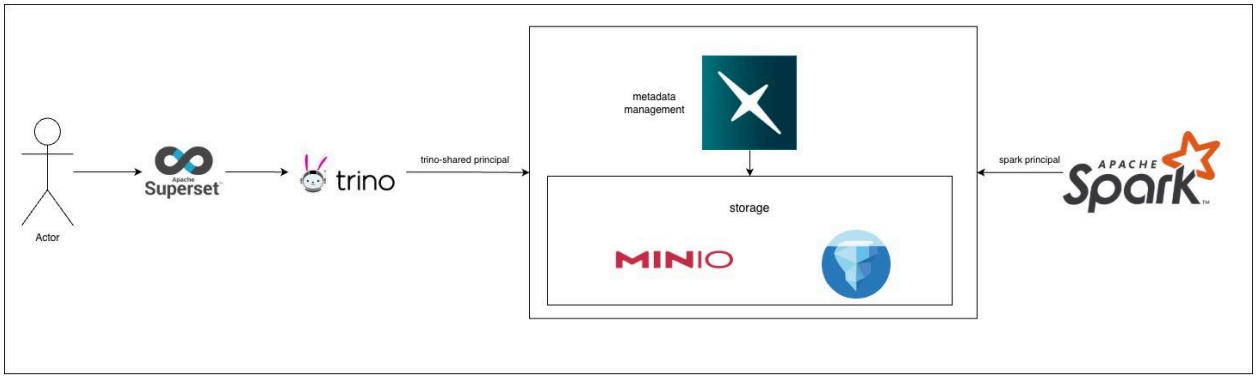
System architecture diagram



As the requirements, while serving analytics purpose query, some users need the data to be up-to-date, doing lakehouse pattern could serve the usage needed as the data can be retrieved as soon as it is written, no hops or reverse ETL.

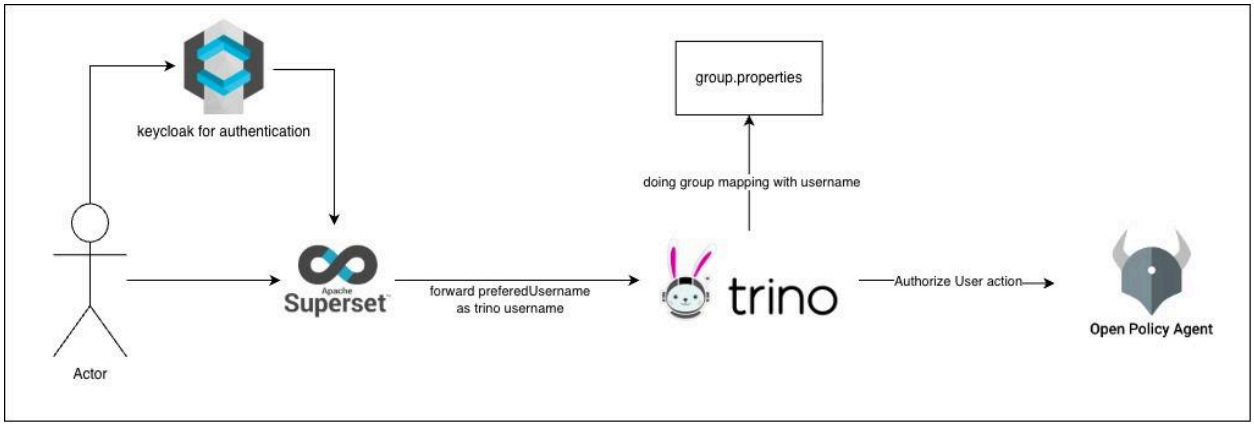
I choose to build this platform fully on an open-source stack because it gives better cost control, flexibility, and scalability in our Kubernetes environment, while avoiding vendor lock-in.

For **storage layer**, I use **MinIO** as the data lake layer. Since we want to use Apache Iceberg, we need object storage underneath. MinIO is S3-compatible and allows the whole platform to run fully self-hosted without cloud dependency.

For the **catalog layer**, I choose **Apache Polaris** as it natively supports Iceberg and provides centralized metadata management shared across engines.

For **query engines**, I separate workloads by **purpose**:
Apache Spark for scheduled ETL and heavy processing
Apache Trino for interactive analytics and ad-hoc queries as the requirement that users come up with many query patterns, for production environments, we can further optimize performance by observing query patterns and applying pre-scaling, caching, or workload tuning based on actual usage behavior.

For **BI and visualization**, I use **Apache Superset** because it integrates well with Trino, supports dashboarding and SQL exploration, and fits well with the open-source ecosystem.



Security model

For **authentication**, I use Keycloak because it integrates well with Superset and Trino through OIDC/OAuth2. In production, it can also integrate with enterprise LDAP or external identity providers.

For **authorization**,I use **Open Policy Agent (OPA)** to handle centralized RBAC and policy-based access control, especially for Trino.

In this diagram, there can still be more improvement.
As the group mapping might not scale well with user grow up, forwarding user identity via superset will be very useful set up as this reference https://github.com/trinodb/trino/issues/28571 so we could even map with another auth service such as ldap or oidc that company already got their desire defined roles.

Guidelines on how to set up and deploy this architecture.

Tooling:
I used Helm packages each component as a chart along with Helmfile to orchestrate all releases in dependency order across namespaces.
For the script, I used Make wraps everything into single commands.

1. Go to infra dir

```
cd problem_2_answer/infra
```

2. Creates a local kind cluster named data-platform.

```
make cluster-up
```

3. Applies Polaris K8s Secrets, builds and loads custom Superset and dbt-spark Docker images into the kind cluster.

```
make pre-sync
```

4. Deploys all Helm releases in dependency order.

```
make sync ENV=local
```

5. Do initial mount polaris to minio bucket

```
make polaris-mount-catalog
```

6. Port-forwards (each in a separate terminal)

```
make port-forward
```

grafana http://localhost:3000
superset http://localhost:8088
keycloak http://localhost:8080

Guidelines on how to perform ETL.

For performing ETL, which should be mainly executed with spark as query engines, I use Spark Operator along with SparkApplication to simplify Spark job deployment and make the platform more scalable on Kubernetes.
I provided 2 example jobs separate in each folder

- init-table : sparkapplication job to create table name raw_food.orders
- load-mock : scheduledsparkapplication job to append data from mock_data into raw_food.orders

Then we can simply
1. Go to directory

```
cd problem_2_answer/application/spark/jobs
```

2. Execute kubectl apply kustomization

```
kubectl apply -k init-table/
kubectl apply -k load-mock/
```

And we get the table along with periodically append data.

```
❯ kubectl exec -it -n query-engines deploy/trino-coordinator -c trino-coordinator -- trino --catalog iceberg --user admin
trino> select * from raw_food.orders;
 order_id | customer_id | amount |  status   | created_at |   _processing_timestamp
----------+-------------+--------+-----------+------------+---------------------------
        1 |         101 | 250.00 | completed | 2024-01-01 | 2026-05-13 17:50:02.342679
        1 |         101 | 250.00 | completed | 2024-01-01 | 2026-05-13 17:49:04.066642
        1 |         101 | 250.00 | completed | 2024-01-01 | 2026-05-13 17:46:59.344600
        1 |         101 | 250.00 | completed | 2024-01-01 | 2026-05-13 17:48:09.628766
        2 |         102 |  89.50 | pending   | 2024-01-02 | 2026-05-13 17:46:59.344600
        3 |         103 | 430.00 | completed | 2024-01-03 | 2026-05-13 17:46:59.344600
        2 |         102 |  89.50 | pending   | 2024-01-02 | 2026-05-13 17:48:09.628766
        3 |         103 | 430.00 | completed | 2024-01-03 | 2026-05-13 17:48:09.628766
        2 |         102 |  89.50 | pending   | 2024-01-02 | 2026-05-13 17:49:04.066642
        3 |         103 | 430.00 | completed | 2024-01-03 | 2026-05-13 17:49:04.066642
        4 |         101 | 120.75 | cancelled | 2024-01-04 | 2026-05-13 17:48:09.628766
        5 |         104 | 670.00 | completed | 2024-01-05 | 2026-05-13 17:48:09.628766
        6 |         105 |  55.00 | pending   | 2024-01-06 | 2026-05-13 17:48:09.628766
        7 |         102 | 310.25 | completed | 2024-01-07 | 2026-05-13 17:48:09.628766
        4 |         101 | 120.75 | cancelled | 2024-01-04 | 2026-05-13 17:49:04.066642
        5 |         104 | 670.00 | completed | 2024-01-05 | 2026-05-13 17:49:04.066642
        6 |         105 |  55.00 | pending   | 2024-01-06 | 2026-05-13 17:49:04.066642
        7 |         102 | 310.25 | completed | 2024-01-07 | 2026-05-13 17:49:04.066642
        4 |         101 | 120.75 | cancelled | 2024-01-04 | 2026-05-13 17:46:59.344600
        5 |         104 | 670.00 | completed | 2024-01-05 | 2026-05-13 17:46:59.344600
        6 |         105 |  55.00 | pending   | 2024-01-06 | 2026-05-13 17:46:59.344600
        7 |         102 | 310.25 | completed | 2024-01-07 | 2026-05-13 17:46:59.344600
        2 |         102 |  89.50 | pending   | 2024-01-02 | 2026-05-13 17:50:02.342679
        3 |         103 | 430.00 | completed | 2024-01-03 | 2026-05-13 17:50:02.342679
        8 |         106 | 980.00 | completed | 2024-01-08 | 2026-05-13 17:46:59.344600
        9 |         103 |  45.00 | cancelled | 2024-01-09 | 2026-05-13 17:46:59.344600
       10 |         107 | 200.00 | pending   | 2024-01-10 | 2026-05-13 17:46:59.344600
       11 |         101 | 750.00 | completed | 2024-01-11 | 2026-05-13 17:46:59.344600
```

This is just a very simple script manipulating little fields for concept, in production we still need a better tool associated with SparkApplication session to handle complex execution requirements and dependencies such as using airflow for orchestrator or use dbt which utilize medallion architecture.

Guidelines on how users can connect to your platform and query the data.

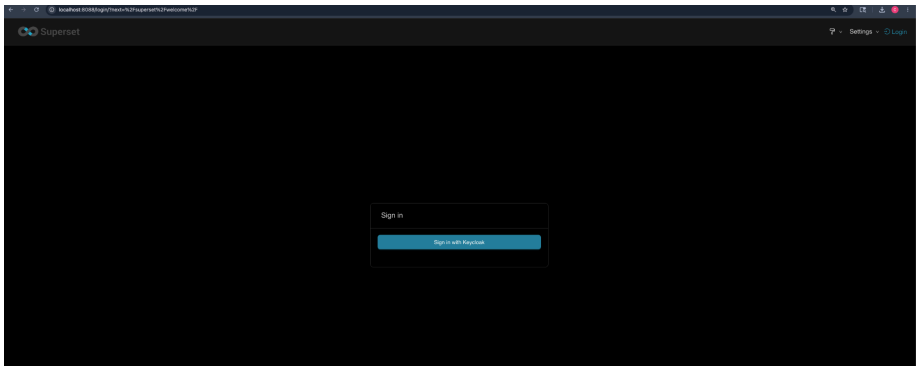
1. In this platform, user can query the data by superset doing oauth2 via keycloak, so for simplify connection via kube, we'll port-forward these 2 components
keycloak

```
kubectl -n auth port-forward svc/keycloak 8080:8080
```

superset

```
kubectl -n bi port-forward svc/superset 8088:8088
```

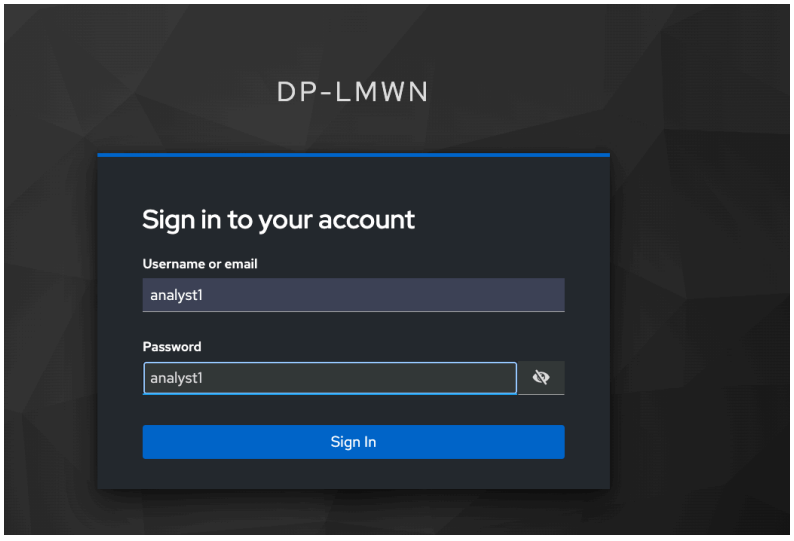
2. Then navigate to localhost:8088, there will be button to sign in with keycloak



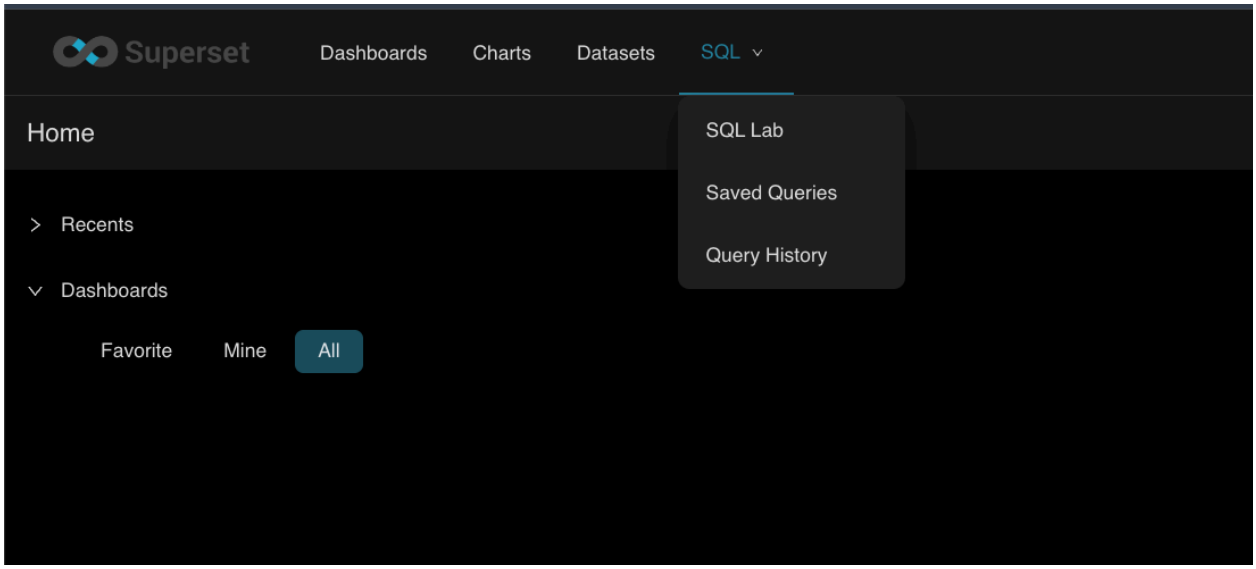
3. For simplify process, there're pre-created username as presentative for each roles

- admin: admin
- analyst1: analyst1
- viewer1: viewer1

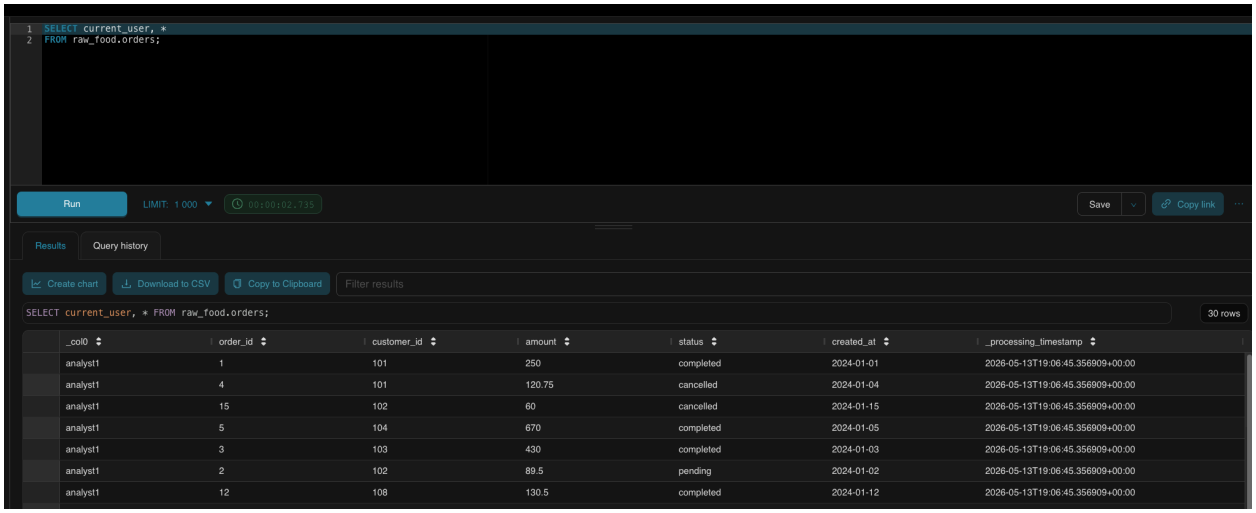
Note that you can also create these user via keycloak UI



4. Here as analyst1, you can queries data with SQL Lab



5. For example, shown data from our ETL job along with username used in Trino



_col0	order_id	customer_id	amount	status	created_at	_processing_timestamp
analyst1	1	101	250	completed	2024-01-01	2026-05-13T19:06:45.356909+00:00
analyst1	4	101	120.75	cancelled	2024-01-04	2026-05-13T19:06:45.356909+00:00
analyst1	15	102	60	cancelled	2024-01-15	2026-05-13T19:06:45.356909+00:00
analyst1	5	104	670	completed	2024-01-05	2026-05-13T19:06:45.356909+00:00
analyst1	3	103	430	completed	2024-01-03	2026-05-13T19:06:45.356909+00:00
analyst1	2	102	89.5	pending	2024-01-02	2026-05-13T19:06:45.356909+00:00
analyst1	12	108	130.5	completed	2024-01-12	2026-05-13T19:06:45.356909+00:00

Going Forward, what can be improve

Storage maintenance
At scale, we also need to monitor and optimize files size and number, by archiving, running vacuum and many various techniques is the one main thing to keep peak performance, and also gives us cost saving. This feature will have a high impact when there is high streaming write into the platforms.

Data Lineage and Catalog
I would pick dbt for batch/micro batch job, dbt artifact give almost everything we need for Data discovery, it lean Analytic engineer process to develop and document with only 1 repo persisting those data into various data source, it gave table lineage for OSS and it now also provided pyspark support. I'm confident that dbt-like will be the new norm and evolve data platforms into data mesh, as we can build many things on top of its artifact.

Data Quality
Dbt also exists in this module, eventually it limits its currently supported sql-based test, dbt there are many great libraries built on top for this feature. However, we need to pick the tool based on data characteristics and test requirements.

Monitoring
Grafana provided support for various metrics types, and many data platform tools implemented with JVM, so adopting this integration would be good.

Security
As mentioned in Security models, the auth forwarding can simplify process and reduce error syncing role for security layers, or we can even implement unified web ui controlling keycloak role + trino group mapping + polaris principal + OPA policies from in on click will help this platform manage RBAC and metadata better.